Project 2: Reinforcement Learning for Quadruped Locomotion

Reinforcement Learning and Optimal Control for Autonomous Systems I

https://github.com/Himhawkins/rob6323_go2_project/tree/master

Pranav Shukla, Jason Bi, Aidan Fitzpatrick
NYU Tandon School of Engineering

1 Introduction

In this project, we train a locomotion policy for the Unitree Go2 quadruped robot in Isaac Lab (a physics simulator) using Proximal Policy Optimization (PPO). PPO is a method of reinforcement learning used to train things through trial and error. It works by running a certain policy, evaluating how 'good' or 'bad' the actions are using rewards, and adjusting the policy accordingly. PPO is quite popular for locomotion-based robots because it can handle continuous actions and is capable of working with neural networks.

In this project, we start from a deliberately weak baseline policy that only rewards linear and yaw velocity tracking, and the goal is to improve gait quality, stability, smoothness, and command tracking through principled reward shaping, regularization, and robustness strategies.

The policy is expected to produce a stable walking or trotting gait with clean footfall timing, maintain a reasonable base height and orientation, and accurately track commanded forward/lateral velocities and yaw rate.

2 Baseline Policy and Limitations

The provided baseline policy showed a very raw model that only rewards velocity tracking. It did not take into consideration any sort of smoothness, robustness, stability, attitude, or height of the gait. This resulted in a rather poor and unpassable simulation. In specific, the baseline policy resulted in the following behaviors:

- The robot exhibited over-excessive pitch/roll of the body, causing a significant tilt in the torso during simulations.
- The robot showed abrupt changes in joint velocity, resulting in inefficient and jerk-like motion.
- The robot appeared to demonstrate dragging of the feet and inconsistent foot clearance from the floor.
- Asymmetric footfall and patterns were observed.
- In addition, high velocity actions that would be detrimental and unrealistic in a real-world quadruped robot.

These limitations led to an unstable form of walking/gait on the quadruped and would not translate well into a real-life Unitree Go2 robot. This was the motivation to adjust and introduce new/improved rewards. Below shows a snapshot of the simulation videos, where we are able to determine efficiency and effectiveness of the gait algorithm:

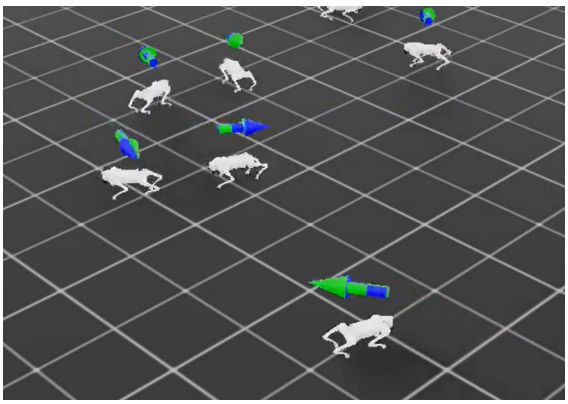


Figure 1: Snapshot of a Simulation Visualizer

3 Methodology

3.1 Training Framework

Training was performed in Isaac Lab using PPO learning. The robot was controlled with randomized forward, lateral velocities and yaw rate, in order to encourage a stronger and more generalized result.

In addition to the baseline control, a manually tuned PD (Proportional-Derivative) controller was used to convert the policy outputs (lateral/forward velocity and yaw rate) into joint torques for actual motor actuation. This control layer streamlined the training by allowing smoother actuation behavior and prevented excessive torque.

3.2 Reward Shaping

To improve gait quality and stability, various reward/penalty terms were implemented into the rob6323_go2_env.py and rob6323_go2_env_cfg.py files:

- Action Smoothness** (rew_action_rate): Penalizes jerky (shaking) motions and sudden action changes and encourages smooth joint movements and lesser torque changes. Scaling Coefficient: -0.1
- Gait Timing: Raibert Heuristic (rew_raibert_heuristic): The Raibert Heuristic is a classic control strategy that places feet to stabilize velocity. We will use it as a "teacher" reward to encourage the policy to learn proper stepping. It penalizes Raibert foot placement heuristic, encouraging feet landing in more stable positions. Scaling Coefficient: -10.0
- Orientation (rew_orient): The orient reward/penalty penalizes the robot body's pitch/roll from the upright standard position, this makes for a more stable body. Scaling Coefficient: -5.0
- Linear Vertical Velocity (rew_lin_vel_z): This addition penalizes vertical velocity, lowering the possibility for 'bouncing' or 'hopping'-like gait, encouraging smoother body movements. Scaling Coefficient: -0.02
- Degree of Freedom Velocity (rew_dof_vel): Penalizes high joint velocities. By discouraging fast, twitchy joint movements, we allow for more efficient motor/actuator use and overall healthier movement for the robot. Scaling Coefficient: -0.0001
- Angular XY Velocity (rew_ang_vel_xy): This penalizes a 'wobbling'-like movement and stabilizes the body. Scaling Coefficient: -0.001
- Foot Clearance (rew_feet_clearance): This prevents toe/foot dragging and cleaner step movements. Scaling Coefficient: -30.0

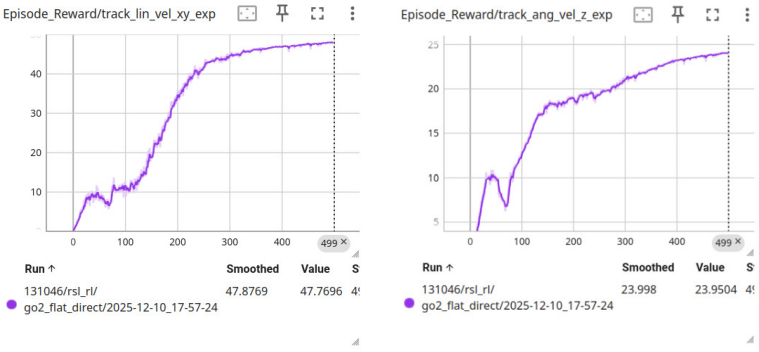
- Contact Reward (rew_tracking_contacts_shaped_force): This encourages more consistent and better distributing of ground contact, improving quality. Scaling Coefficient: 4.0

Each modification was introduced incrementally and evaluated independently. In addition, a base_height_min = 0.05 was implememted, to set a minimum distance the base of the robot could have to the ground, to prevent unrealistic dynamics and inefficient movements.

4 Results (with joint friction)

4.1 Quantitative Metrics

Performance was evaluated using metrics recorded with Tensorbaord visualizer during training. In the figures below, the evolution of each reward (vs number of runs) can be seen:



(a) Linear velocity tracking (xy) training reward vs runs. It can be seen to converge to the target value of 48

(b) Angular velocity tracking (z) training reward vs runs. It can be seen to converge to the target value of 24

Figure 2: Velocity command tracking performance during training.

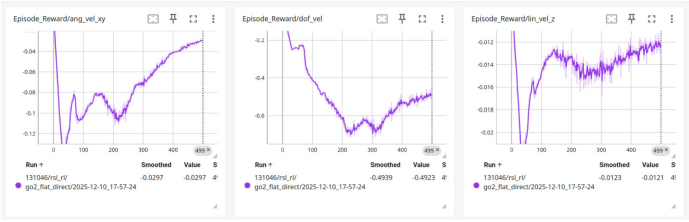


Figure 3: Additional Reward Plots (Angular XY, DOF, and Linear Z Velocity)

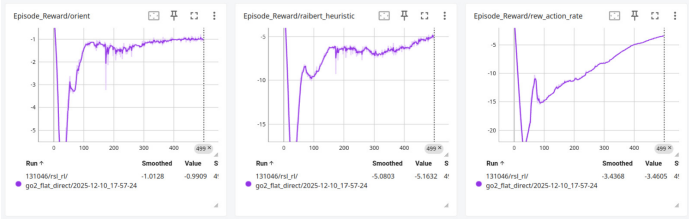


Figure 4: Additional Reward Plots (Orientation, Raibert Hueristic, and Action Smoothness)

Figure 5: Additional Reward Plots (Foot Clearance and Contacts)

4.2 Qualitative Evaluation

Figure 6 shows a snapshot of the final simulation obtained with the finalized rewards, control, and constraint tuning. The .mp4 file can be viewed in our team's BrightSpace submission, separately:

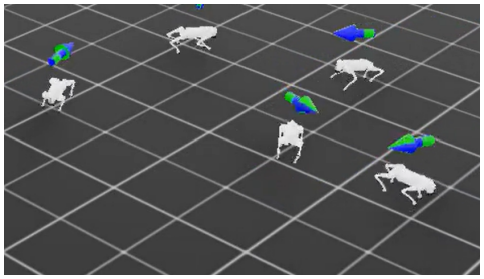


Figure 6: Snapshot of the final walking gait simulation.

5 Discussion

The final policy we landed upon demonstrates that carefully integrated reward shaping and constraint tuning can vastly improve quadruped locomotion and movement without explicitly programming gait dynamics. Implementing smoothness rewards, foot clearance, gait-timing, velocity, orientation, and contact-oriented rewards and penalties led us to a more stable and realistic quadruped simulation. It was important, however, to tune the reward coefficients with careful experimentation, leading to different patterns/performances.

6 Conclusion

This project demonstrates how reward shaping and regularization of an intentionally weak quadruped gait policy can drastically improve its function and efficiency. By tweaking and implemeting new rewards related to efficiency, action smoothness, contact parameters, and foot clearances, the new policy can demonstrate improved gait and command.

7 BONUS

7.1 Actuator Friction Model with Randomization

7.2 New Skill

We implemented randomised friction in the actuator by subtracting a calculated coulomb friction force equilvalent from the torque commands. Implementation:

```python
def _apply_action(self) -> None:

    # Compute PD torques
    torques = torch.clip(
        (
            self.Kp * (
                self.desired_joint_pos
                - self.robot.data.joint_pos
            )
            - self.Kd * self.robot.data.joint_vel
        ),
        -self.torque_limits,
        self.torque_limits,
    )

    # Modelling Joint Friction!
    q_dot=self.robot.data.joint_vel
    tau_stiction = self.F_s * torch.tanh(q_dot / 0.1)
    tau_viscous = self.mu_v * q_dot
    tau_friction = tau_stiction + tau_viscous
    torques=torques-tau_friction
    # Apply torques to the robot
    self.robot.set_joint_effort_target(torques)
```

```python
def _reset_idx(self, env_ids: Sequence[int] | None):

    dist_mu_v = Uniform(0.0, 0.3)
    dist_F_s = Uniform(0.0, 2.5)

    # 2. Sample single values (returns a scalar tensor)
    self.mu_v[env_ids,:] = dist_mu_v.sample((len(env_ids),12)).to(self.device)
    self.F_s[env_ids,:] = dist_F_s.sample((len(env_ids),12)).to(self.device)
```

Figure 7: Implementation of bonus Joint Friction

References

1. Joseph Amigo, Rooholla Khorrambakht, Elliot Chane-Sane, Ludovic Righetti, and Nicolas Mansard. *First order model-based RL through decoupled backpropagation*. In Conference on Robot Learning (CoRL), 2025.

2. Elliot Chane-Sane, Joseph Amigo, Thomas Flayols, Ludovic Righetti, and Nicolas Mansard. *SoloParkour: Constrained reinforcement learning for visual locomotion from privileged experience*. In Conference on Robot Learning (CoRL), 2024.